

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Алтайский государственный технический университет им. И. И. Ползунова»

Факультет информационных технологий Кафедра информатики,
вычислительной техники и информационной безопасности

Отчет защищен с оценкой _____
Преподаватель _____ С. В. Умбетов
« ____ » _____ 2026 г.

**Отчёт по лабораторной работе №1 по дисциплине
«Устройство и функционирование интеллектуальных
информационных систем» «работы с табличными
данными в библиотеке Pandas»**

ЛР 09.02.08.21.001

Студент группы <u>1ИИС-41</u>	<u>Т.Е.Рябич</u>
группа	и.о., фамилия
Преподаватель <u>доцент, к. т. н.</u>	<u>С. В. Умбетов</u>
должность, ученая степень	и.о., фамилия

БАРНАУЛ 2026

Цель работы: освоить базовые и продвинутые приёмы работы с табличными данными в библиотеке Pandas на примере реального набора данных о пассажирах «Титаника», научиться загружать, исследовать, фильтровать, агрегировать и преобразовывать табличные данные, а также делать аналитические выводы.

Задачи работы: изучить основы Pandas (Series и DataFrame), освоить чтение CSV-файлов, научиться получать первичную информацию о датафрейме, освоить индексацию, фильтрацию по условиям, применение статистических методов, группировку и агрегацию данных, построение сводных таблиц, обработку пропущенных значений и дубликатов, создание новых признаков, а также подготовить аналитические выводы и ответы на контрольные вопросы.

Задание к работе: выполнить последовательно 12 практических задач по работе с датасетом train.csv с применением инструментов библиотеки Pandas, составить аналитический отчёт по результатам и ответить на теоретические контрольные вопросы.

Задание принято: _____



Подпись

Рябич Т.Е.

Ход работы

Задача 1. Загрузка и первичный осмотр данных

```
titanic = pd.read_csv('train.csv')
display(titanic.head(10))
display(titanic.tail(5))
```

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4583	NaN	Q
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0750	NaN	S
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1333	NaN	S
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30.0708	NaN	C
PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.00	NaN	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.00	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.45	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.00	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.75	NaN	Q

Рисунок 1 – Загрузка датасета train.csv, вывод первых 10 и последних 5 строк таблицы.

Задача 2. Получение информации о датафрейме

```
titanic.info()
display(titanic.describe())
display(titanic.describe(include=['object']))
```

```
*** <class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

	Name	Sex	Ticket	Cabin	Embarked	
count	891	891	891	204	889	
unique	891	2	681	147	3	
top	Dooley, Mr. Patrick	male	347082	G6	S	
freq		1	577	7	4	644

Рисунок 2 – Получение общей технической сводки и статистических характеристик датафрейма.

Задача 3. Работа с Series

```
embarked_series = titanic['Embarked'].head(5).copy()
embarked_series.index = ['passenger1', 'passenger2', 'passenger3', 'passenger4', 'passenger5']
print(embarked_series.loc['passenger3'])
print(embarked_series.iloc[2])
```

```
*** S
S
```

Рисунок 3 – Создание объекта Series из столбца Embarked, изменение индексов и проверка методов доступа .loc и .iloc.

Задача 4. Статистические методы

```

mean_age = titanic['Age'].mean()
std_age = titanic['Age'].std()
min_fare = titanic['Fare'].min()
max_fare = titanic['Fare'].max()

print(f"Средний возраст: {mean_age}")
print(f"Стандартное отклонение возраста: {std_age}")
print(f"Минимальная стоимость: {min_fare}")
print(f"Максимальная стоимость: {max_fare}")

Средний возраст: 29.69911764705882
Стандартное отклонение возраста: 14.526497332334044
Минимальная стоимость: 0.0
Максимальная стоимость: 512.3292

```

Рисунок 4 – Вычисление базовых статистических параметров (среднее, стандартное отклонение, минимум, максимум).

Задача 5. Фильтрация данных по условиям

```

df_age_30 = titanic[titanic['Age'] > 30]
df_fare_above_mean = titanic[titanic['Fare'] > titanic['Fare'].mean()]
df_pclass1_female = titanic[(titanic['Pclass'] == 1) & (titanic['Sex'] == 'female')]
mean_age_pclass1_female = df_pclass1_female['Age'].mean()

print(f"Средний возраст женщин 1-го класса: {mean_age_pclass1_female}")

Средний возраст женщин 1-го класса: 34.61176470588235

```

Рисунок 5 – Фильтрация датафрейма по одиночным условиям и расчет среднего возраста женщин из первого класса.

Задача 6. Комбинированная фильтрация и статистика

```

survived_children = titanic[(titanic['Survived'] == 1) & (titanic['Age'] < 18)]
count_children = len(survived_children)
mean_fare_children = survived_children['Fare'].mean()

print(f"Количество выживших детей: {count_children}")
print(f"Средняя стоимость их билета: {mean_fare_children}")

Количество выживших детей: 61
Средняя стоимость их билета: 34.857309836065575

```

Рисунок 6 – Выделение подгруппы выживших несовершеннолетних пассажиров с расчетом метрик.

Задача 7. Группировка данных

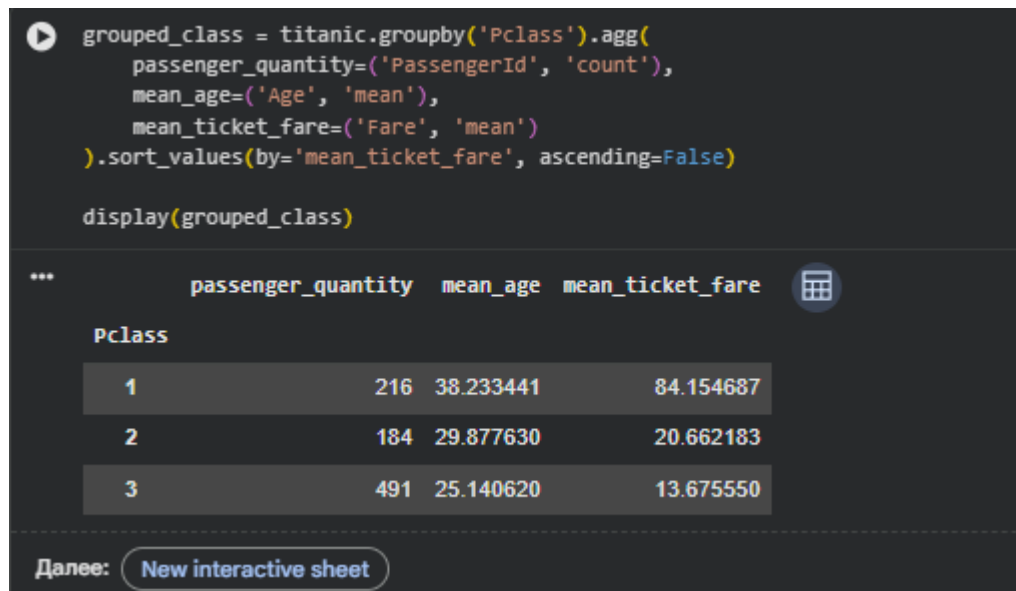


Рисунок 7 – Группировка пассажиров по классам билетов с агрегацией и сортировкой.

Задача 8. Сводная таблица

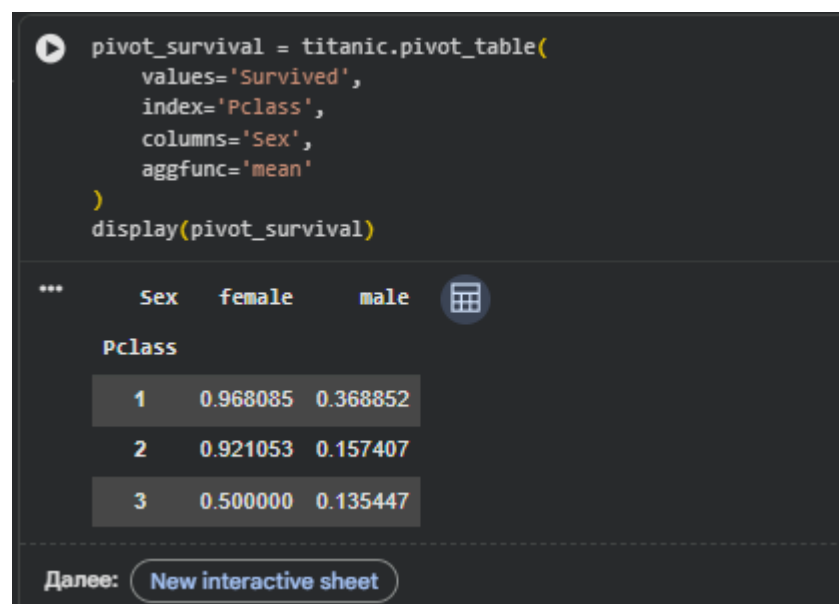


Рисунок 8 – Построение сводной таблицы выживаемости пассажиров в зависимости от пола и класса билета.

Данные наглядно подтверждают правило спасения «сначала женщины и дети». Наивысшие шансы на спасение зафиксированы у женщин из первого класса (около 97%), в то время как самая низкая выживаемость наблюдается у мужчин из третьего класса (всего 13.5%).

Задача 9. Работа с пропущенными значениями

```
display(titanic.isnull().sum())
null_percent = (titanic.isnull().sum() / len(titanic)) * 100
display(null_percent)
titanic['Embarked'] = titanic['Embarked'].fillna('S')
titanic['Age'] = titanic['Age'].fillna(titanic['Age'].median())
print("дубликатов:", titanic.duplicated(subset=['PassengerId']).sum())
titanic = titanic.drop_duplicates(subset=['PassengerId'], keep='first')
titanic.info()
```

	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2

dtype: int64

Рисунок 9 – Анализ пропущенных данных, их заполнение и очистка таблицы от дубликатов.

PassengerId	0.000000
Survived	0.000000
Pclass	0.000000
Name	0.000000
Sex	0.000000
Age	19.865320
SibSp	0.000000
Parch	0.000000
Ticket	0.000000
Fare	0.000000
Cabin	77.104377
Embarked	0.224467

dtype: float64
Дубликатов: 0
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
Column Non-Null Count Dtype
--- ---
0 PassengerId 891 non-null int64
1 Survived 891 non-null int64
2 Pclass 891 non-null int64
3 Name 891 non-null object
4 Sex 891 non-null object
5 Age 891 non-null float64
6 SibSp 891 non-null int64
7 Parch 891 non-null int64
8 Ticket 891 non-null object
9 Fare 891 non-null float64
10 Cabin 204 non-null object
11 Embarked 891 non-null object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB

Рисунок 10 – Анализ пропущенных данных, их заполнение и очистка таблицы от дубликатов.

Задача 10. Создание новых признаков и расширенный анализ

```
titanic['FamilySize'] = titanic['SibSp'] + titanic['Parch'] + 1
titanic['IsAlone'] = titanic['FamilySize'].apply(lambda x: 1 if x == 1 else 0)

titanic['AgeGroup'] = pd.cut(
    titanic['Age'],
    bins=[0, 12, 18, 35, 60, 100],
    labels=['child', 'teenager', 'young', 'adult', 'old']
)

age_analysis = titanic.pivot_table(
    values=['Survived', 'Fare', 'PassengerId'],
    index='AgeGroup',
    aggfunc={'Survived': 'mean', 'Fare': 'mean', 'PassengerId': 'count'}
).rename(columns={'PassengerId': 'Quantity'})

display(age_analysis)
```

... /tmp/ipykernel_2796/678284358.py:10: FutureWarning: The default value of observed=False is deprecated

	Fare	Quantity	Survived
AgeGroup			
child	31.536352	69	0.579710
teenager	33.451313	70	0.428571
young	27.356166	535	0.353271
adult	44.259658	195	0.400000
old	41.371214	22	0.227273

Далее: [New interactive sheet](#)

Рисунок 11 – Генерация новых признаков, разбиение на возрастные категории и сводный анализ выживаемости.

Максимальный показатель выживаемости наблюдается в группе «child» (58%), поскольку их эвакуировали в приоритетном порядке. Меньше всего выживали люди из категории «old» (22%).

Задача 11. Комплексный аналитический отчёт



Рисунок 12 – Анализ характеристик пассажиров в разрезе портов посадки.

Пассажиры, севшие на борт в порту С (Шербур), выживали чаще всего. Статистика показывает, что из этого порта села наибольшая относительная доля пассажиров самого дорогого первого класса, что обеспечило им приоритет при эвакуации.

Задача 12. Сравнительный анализ по размеру семьи

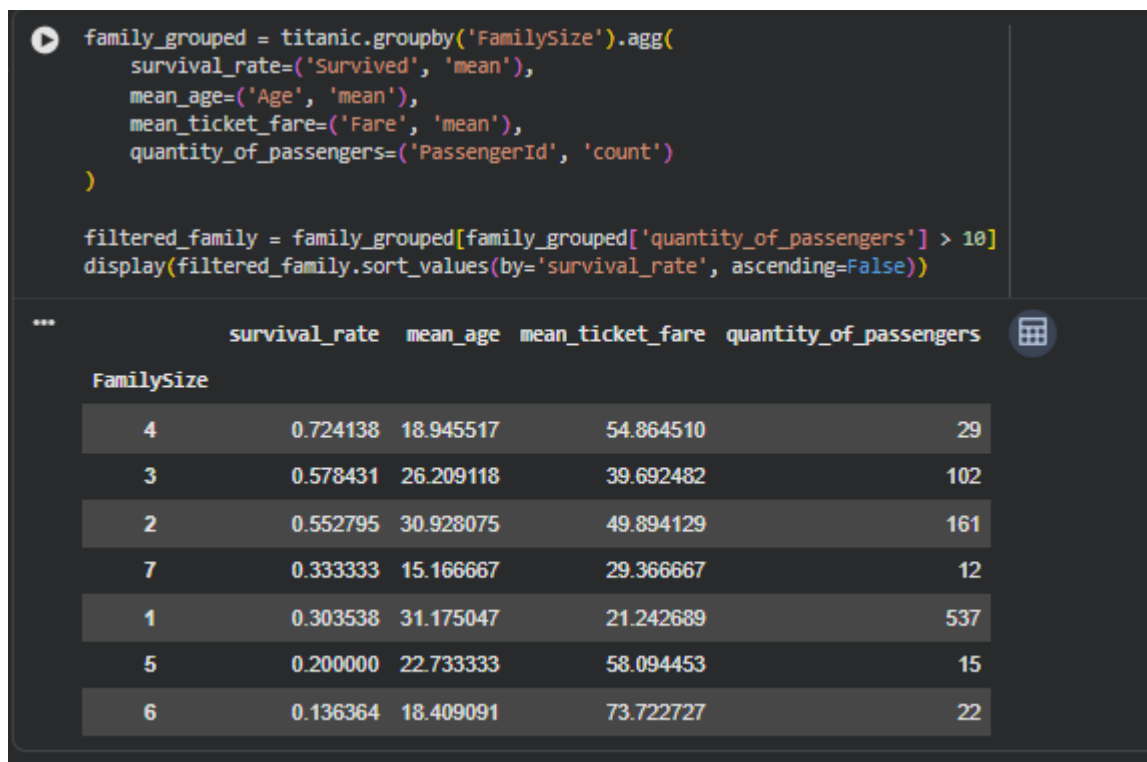


Рисунок 13 – Расчет и сравнение показателей выживаемости в зависимости от количества членов семьи на борту.

Наибольшие шансы на выживание (до 72%) имели небольшие семьи, состоящие из 2–4 человек. Одинокие пассажиры, а также члены крупных многодетных семей

(от 5 человек и более) погибали значительно чаще из-за трудностей сбора всех родственников во время паники и нехватки мест в шлюпках.

Вывод

В ходе лабораторной работы были освоены базовые и продвинутое методы манипуляции с таблицами с помощью библиотеки Pandas на примере реального датасета «Титаник». Были закреплены навыки импорта данных, проведения первичного статистического анализа, применения условий фильтрации, агрегации данных через группировки и сводные таблицы, а также заполнения пропусков и генерации новых аналитических признаков. На основе расчетов сформированы обоснованные выводы о факторах, напрямую влиявших на выживаемость пассажиров в катастрофе.

Контрольные вопросы

- 1. Что такое Pandas и для каких задач используется эта библиотека?**
Это высокоуровневая библиотека Python для анализа, очистки, трансформации и структурирования табличных данных. Используется для загрузки файлов, фильтрации, устранения пропусков и расчета статистических метрик.
- 2. Чем Series отличается от обычного списка Python? Как создать Series с пользовательским индексом?**
В Series каждому элементу жестко присвоен явный (в том числе текстовый) индекс, и все элементы должны принадлежать к одному типу данных. Создается кодом: `pd.Series(данные, index=[список_индексов])`.
- 3. В чём разница между .loc и .iloc при доступе к элементам Series?**
Метод `.loc` осуществляет поиск и извлечение данных по их текстовым меткам и названиям индексов. Метод `.iloc` работает строго по системным порядковым номерам позиций от 0, игнорируя текст.
- 4. Как создать DataFrame из словаря? Как задать названия колонок и индекс вручную?**
Нужно передать словарь в конструктор `pd.DataFrame(словарь)`, ключи станут именами колонок. Вручную названия столбцов и строк настраиваются параметрами `columns` и `index` в том же конструкторе.
- 5. Какие параметры функции pd.read_csv() вы знаете? Для чего используется параметр usecols?**
Основные параметры: путь к файлу, разделитель `sep`, разделитель дроби `decimal` и обработка дат `parse_dates`. Параметр `usecols` загружает только выбранные столбцы, экономя оперативную память.
- 6. Какую информацию выводит метод .info()? Какие параметры можно ему передать?**
Выводит число строк, названия и типы колонок, объем занятой памяти и количество непустых заполненных ячеек. Можно передать `verbose` для детальности или `memory_usage` для управления отображением памяти.

7. Что показывает метод `.describe()` для числовых данных и для текстовых данных?

Для чисел считает количество, среднее, отклонение, минимум, максимум и квартили. Для текста при параметре `include=['object']` выдает число строк, количество уникальных значений, моду и ее частоту.

8. Как получить доступ к отдельному столбцу `DataFrame`? Какие два способа существуют?

Первый способ — указать название столбца строкой внутри квадратных скобок `df['col_name']`. Второй способ — обратиться через точку как к свойству объекта `df.col_name` (работает, если в имени нет пробелов).

9. Как отфильтровать `DataFrame` по одному условию? Как комбинировать несколько условий?

Для одного условия логическое выражение помещают внутри квадратных скобок датафрейма. Несколько условий объединяют знаками `&` (И) или `|` (ИЛИ), обязательно оборачивая каждое отдельное условие в круглые скобки.

10. Какие статистические методы можно применять к `DataFrame` и `Series`? Что произойдет, если применить `.mean()` ко всему `DataFrame` с текстовыми столбцами?

Применяют методы суммы `.sum()`, среднего `.mean()`, минимума/максимума и отклонения `.std()`. Современные версии Pandas на текстовых столбцах выдадут ошибку, поэтому важно явно добавлять параметр `numeric_only=True`.

11. Для чего используется метод `.groupby()`? Как применить несколько агрегирующих функций одновременно?

Используется для группировки строк по повторяющимся значениям в указанных колонках. Для расчета нескольких метрик сразу после группировки вызывают метод `.agg()`, куда передают список нужных функций в скобках.

12. Чем сводная таблица (`pivot_table`) отличается от обычной группировки? В каких случаях удобнее использовать каждую из них?

Группировка `.groupby()` возвращает плоский линейный `Series` или `DataFrame`. Сводная таблица строит двумерную сетку (строки и столбцы). Группировка удобна для простых подсчетов, сводная таблица — для выявления кросс-зависимостей двух признаков.

13. Как обнаружить и обработать пропущенные значения в `DataFrame`? Какие стратегии заполнения пропусков вы знаете?

Находят пропуски связкой `.isnull().sum()`. Стратегии включают полное удаление строк с пропусками через `.dropna()`, заполнение константой или вычисленной статистикой (медианой, средним) через `.fillna()`.

14. Как создать новый признак на основе существующих? Для чего используется метод `pd.cut()`?

Новый признак создается через математические операции со столбцами или вызовом `.apply()` с `lambda`-функцией. Метод `pd.cut()` нужен для

оцифровки непрерывных числовых данных в заданные интервалы-категории.

15. Как отсортировать DataFrame по значениям одного или нескольких столбцов? Как вывести только топ-N строк?

Сортируют методом `.sort_values(by='имя_столбца')`, регулируя направление параметром `ascending`. Чтобы оставить только топ-N строк, к результату сортировки сразу применяют метод `.head(N)`.